

# A01: Richiami di metodi numerici per modelli stocastici

Antonio Scala

10 Mar 2026

Questa appendice richiama strumenti numerici di base che ricorrono continuamente in simulazioni e stima di parametri: ricerca di zeri, ottimizzazione (anche vincolata), risoluzione di sistemi lineari, condizionamento, criteri di arresto e diagnostica di fallimenti. L'obiettivo non è l'eshaustività, ma una “cassetta degli attrezzi” concreta, con enfasi su robustezza, controlli e patologie tipiche.

## 1. Ricerca di zeri

### 1.1 Problema

Dato  $f : \mathbb{R} \rightarrow \mathbb{R}$ , trovare  $x^*$  tale che

$$f(x^*) = 0.$$

Caso tipico: inversione di una relazione implicita, equilibrio stazionario, vincoli di consistenza.

### 1.2 Metodo di bisezione (bracketing, robusto)

Se esistono  $a < b$  con  $f(a)f(b) < 0$ , allora per continuità esiste almeno uno zero in  $(a, b)$  (teorema degli zeri). La bisezione costruisce una successione di intervalli annidati che preserva il cambio di segno.

**Algoritmo (idea):** - porre  $m = (a + b)/2$ ; - se  $f(a)f(m) < 0$ , sostituire  $b \leftarrow m$ , altrimenti  $a \leftarrow m$ ; - ripetere finché  $|b - a|$  è piccolo.

**Pro:** - converge sempre (se ipotesi rispettate); - controllo diretto dell'errore: dopo  $k$  iterazioni, ampiezza intervallo  $(b - a)/2^k$ .

**Contro:** - convergenza lenta (lineare); - richiede un bracket valido (cambio di segno).

### 1.3 Metodo di Newton (rapido, ma fragile)

Assumendo  $f$  derivabile e  $f'(x) \neq 0$  vicino allo zero:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}.$$

**Pro:** - convergenza tipicamente quadratica vicino alla soluzione.

**Contro (patologie tipiche):** - divergenza se  $x_0$  è “cattivo”; - stagnazione/cicli (es. 2-ciclo) anche su funzioni semplici; - fallisce se  $f'(x_n) \approx 0$  (passo enorme) o se  $f'$  non è affidabile.

### 1.4 Metodi “ibridi” e derivative-free per zeri

- **Secante:** sostituisce  $f'(x_n)$  con una derivata numerica da due punti; è più robusto di Newton quando la derivata è difficile, ma non ne elimina i problemi.
- **Bisezione + Newton:** pratica standard: si mantiene un bracket e si tenta un passo di Newton; se esce dall'intervallo o peggiora, si ripiega su bisezione. Questo unisce robustezza e velocità.

### 1.5 Diagnostica rapida per zeri

Quando un metodo “non va”: - controllare che esista davvero uno zero nel dominio considerato; - visualizzare  $f(x)$  in un intorno (anche grossolanamente); - monitorare: - residuo  $|f(x_n)|$ , - passo  $|x_{n+1} - x_n|$ , - numero iterazioni, - eventuali cicli (ripetizione di valori).

## 2. Ricerca di minimi/massimi (ottimizzazione non vincolata)

### 2.1 Problema

Dato  $f : \mathbb{R}^d \rightarrow \mathbb{R}$ , trovare

$$x^* \in \arg \min_x f(x).$$

Per massimi, si minimizza  $-f$ .

### 2.2 Condizioni del primo e secondo ordine

Se  $f$  è differenziabile, un candidato minimo è un *punto estremo*  $x^*$  che soddisfi

$$\nabla f(x^*) = 0.$$

Se  $f$  è due volte differenziabile, un minimo locale “stretto” ha Hessiano  $H(x^*)$  definito positivo.

## 2.3 Discesa del gradiente (essenziale, ma non miracolosa)

Iterazione:

$$x_{n+1} = x_n - \alpha_n \nabla f(x_n).$$

Con  $\alpha_n$  (step size) costante o scelto via una semplice *line search*<sup>1</sup>.

**Pro:** - concettualmente semplice; - scala bene in dimensione.

**Contro:** - scelta di  $\alpha_n$  cruciale: troppo grande diverge, troppo piccolo stagnazione; - converge a minimi locali in funzioni non convesse; - sensibile a scalatura e condizionamento.

## 2.4 Metodi “Newton-like” per minimi (idea)

Newton per ottimizzazione:

$$x_{n+1} = x_n - H^{-1}(x_n) \nabla f(x_n),$$

che richiede risolvere un sistema lineare<sup>2</sup>. In pratica si usano varianti (quasi-Newton, line search, trust region) per robustezza.

## 3. Sistemi lineari e condizionamento

### 3.1 Problema

Risolvere

$$Ax = b,$$

---

<sup>1</sup>Una *line search* sceglie automaticamente lo step  $\alpha_n$  lungo la direzione di discesa  $p_n = -\nabla f(x_n)$ , cercando un compromesso tra “fare passi grandi” e mantenere la diminuzione di  $f$ . In pratica si considera la funzione monodimensionale

$$\phi(\alpha) = f(x_n + \alpha p_n)$$

e si cerca un  $\alpha_n > 0$  tale che  $f(x_n + \alpha_n p_n) < f(x_n)$  in modo affidabile. La versione più semplice e robusta è il **backtracking**: si parte da un valore iniziale  $\alpha = \alpha_0$  (ad esempio  $\alpha_0 = 1$ ), e lo si riduce geometricamente  $\alpha \leftarrow \beta \alpha$  con  $0 < \beta < 1$  finché la condizione di **diminuzione sufficiente** (Armijo) è soddisfatta,

$$f(x_n + \alpha p_n) \leq f(x_n) + c \alpha \nabla f(x_n) \cdot p_n,$$

con  $c \in (0, 1)$  piccolo. Questo evita passi troppo grandi che farebbero aumentare  $f$ , senza dover minimizzare  $\phi(\alpha)$  in modo accurato ad ogni iterazione.

<sup>2</sup>L'idea di  $H^{-1}(x_n)$  nasce dall'approssimazione locale di  $f$  tramite Taylor al secondo ordine attorno a  $x_n$ :

$$f(x_n + p) \approx f(x_n) + \nabla f(x_n) \cdot p + \frac{1}{2} p^T H(x_n) p.$$

Se  $H$  è definita positiva (per cui  $p^T H p$  cresce come  $p^2$ ), minimizzando questa forma quadratica rispetto a  $p$  (sto cercando una  $f(x_n + p)$  “piccola”) si ottiene la condizione del primo ordine

$$\nabla f(x_n) + H(x_n) p = 0,$$

da cui

$$p = -H(x_n)^{-1} \nabla f(x_n),$$

cioè il passo di Newton: l'inverso dell'Hessiano “scala e ruota” il gradiente tenendo conto della curvatura locale (direzioni ripide vs piatte).

con  $A \in \mathbb{R}^{d \times d}$ .

Appare in: - Newton per zeri in  $d$  dimensioni:  $J(x)\delta = -F(x)$ ; - Newton per minimi:  $H(x)\delta = -\nabla f(x)$ ; - regressione ai minimi quadrati, stima di parametri, ecc.

### 3.2 Condizionamento (perché “numericamente” può esplodere)

Se  $A$  è mal condizionata, piccoli errori in  $b$  o round-off possono produrre grandi errori in  $x$ . Una misura classica è il numero di condizionamento (in norma-2  $\|A\|_2 = \max_{x \neq 0} \frac{\|Ax\|_2}{\|x\|_2}$ ):

$$\kappa_2(A) = \|A\|_2 \|A^{-1}\|_2.$$

**Regola pratica:** -  $\kappa(A)$  elevato implica amplificazione dell'errore; - riscalarle delle variabili (unità, normalizzazione) può migliorare drasticamente la situazione.

### 3.3 Sanity checks per sistemi lineari

- **Controllo del residuo:** dopo aver calcolato  $x$ , verifica quanto bene soddisfa il sistema. Calcola  $r = b - Ax$  e guarda  $\|r\|$  (meglio anche relativo): se  $\|r\|$  non è piccolo rispetto a  $\|b\|$ , la soluzione non è affidabile (o l'algoritmo non ha davvero risolto il sistema).
- **Ordini di grandezza e scaling:** controlla se le componenti di  $x$  hanno scale “ragionevoli” dato  $A$  e  $b$ . Se alcune componenti esplodono o differiscono di molti ordini di grandezza senza motivo fisico/modellistico, spesso c'è un problema di scalatura o condizionamento.
- **Sensibilità a perturbazioni:** modifica leggermente  $b \rightarrow b + \delta b$  (anche rumore piccolo) e risolvi di nuovo. Se  $x$  cambia molto rispetto alla perturbazione, il sistema è mal condizionato: la soluzione può avere un residuo piccolo ma essere comunque molto instabile.

## 4. Criteri di arresto e diagnostica del fallimento (trasversale)

Ogni algoritmo iterativo deve dichiarare esplicitamente: - quando si ferma; - con quale “qualità” del risultato; - come segnala fallimento.

### 4.1 Residuo vs incremento

Due indicatori diversi: - **residuo:** quanto bene soddisfa l'equazione/condizione (es.  $|f(x_n)|$  o  $\|\nabla f(x_n)\|$ ); - **incremento:** quanto sta cambiando la soluzione (es.  $\|x_{n+1} - x_n\|$ ).

Si può avere: - incremento piccolo ma residuo non piccolo (stagnazione); - residuo piccolo ma incremento non piccolo (instabilità numerica o oscillazioni).

## 4.2 Tolleranze assolute e relative

Tipicamente si usa una combinazione:

$$\text{stop se } |f(x_n)| \leq \text{atol} + \text{rtol} |f(x_0)|.$$

Analogamente per gradienti e passi.

## 4.3 Diagnostica minima da stampare/loggare

Per ogni iterazione (o ogni  $k$  iterazioni): - iterazione  $n$ ; -  $x_n$ ; - residuo; - passo; - messaggio di uscita (convergenza, max iter, divergenza, ciclo sospetto).

## 5. Minimi locali e multi-start

### 5.1 Perché succede

Se  $f$  non è convessa, esistono più minimi locali: l'algoritmo converge tipicamente al minimo nel bacino di attrazione determinato dall'inizializzazione.

### 5.2 Strategia multi-start (economica e spesso efficace)

- scegliere una lista di punti iniziali (griglia grossolana o campionamento casuale);
- lanciare l'ottimizzatore da ciascun punto;
- selezionare la miglior soluzione trovata (con controlli di convergenza).

**Nota didattica:** multi-start non “garantisce” di trovare ottimi globali, ma rende visibile la dipendenza dall'inizializzazione e spesso migliora risultati in pratica.

## 6. Metodi *derivative-free* essenziali

Quando derivate non sono disponibili, sono rumorose, o Newton/gradiente falliscono:

### 6.1 Zeri in 1D <sup>3</sup>

- bisezione (richiede bracket);
- secante (non richiede derivata, ma è più fragile).

---

<sup>3</sup>Per  $F : \mathbb{R}^d \rightarrow \mathbb{R}^d$  il problema “trovare uno zero” significa trovare  $x^*$  tale che  $F(x^*) = 0$  (cioè un sistema di  $d$  equazioni in  $d$  incognite). In  $d > 1$  non esiste un equivalente diretto della bisezione: in 1D puoi ordinare i punti su una retta e usare il cambio di segno per “intrappolare” lo zero; in dimensione maggiore manca questo ordinamento e uno zero non è, in generale, qualcosa che puoi racchiudere con due estremi. In più, se lo Jacobiano  $J(x^*)$  è singolare (determinante nullo), lo zero può non essere **isolato**: l'insieme delle soluzioni può essere una curva/superficie, e allora i metodi tipo Newton possono diventare instabili o non univoci (ci sono infinite direzioni compatibili). Per questo, in  $d > 1$  si usano metodi basati su linearizzazioni (Newton/quasi-Newton) e servono ipotesi di regolarità (tipicamente  $J$  invertibile) per avere convergenza “pulita”.

## 6.2 Minimi in 1D

- ricerca su intervallo (golden section) per funzioni unimodali;
- metodi ibridi (tipo Brent) combinano robustezza e velocità.

## 6.3 Minimi in dimensione maggiore (idea)

- coordinate descent (semplice, ma può essere lento);
- pattern search / Nelder–Mead (euristici, attenzione a diagnosi e scalatura).

## 7. Ottimizzazione vincolata

### 7.1 Problema (formulazione standard “s.t.”)

In ottimizzazione si scrive tipicamente un problema vincolato nella forma:

$$\begin{aligned} \min_{x \in \mathbb{R}^d} \quad & f(x) \\ \text{s.t.} \quad & h_j(x) = 0, \quad j = 1, \dots, m, \\ & g_i(x) \leq 0, \quad i = 1, \dots, p. \end{aligned}$$

dove: -  $f(x)$  è la **funzione obiettivo** (cio che vuoi minimizzare); -  $h_j(x) = 0$  sono i **vincoli di uguaglianza**; -  $g_i(x) \leq 0$  sono i **vincoli di disuguaglianza**;  
- “s.t.” significa **subject to** (soggetto a / tali che).

Un vincolo **box** si scrive spesso esplicitamente come

$$a \leq x \leq b,$$

ma in forma “s.t.” è equivalente a due disuguaglianze componente per componente:

$$x - a \geq 0, \quad b - x \geq 0,$$

oppure, usando solo il formato  $g(x) \leq 0$ :

$$a - x \leq 0, \quad x - b \leq 0.$$

Questo è di fatto il formato che “si da in pasto” ai programmi di ottimizzazione industriali: un obiettivo ed un insieme di vincoli (uguaglianze/disuguaglianze) che definiscono l’insieme ammissibile.

### 7.2 Tre ricette pratiche: penalita, barriera, proiezione

(i) **Penalita (penalty methods)** Trasformare in non vincolato aggiungendo un costo per violazione:

$$\tilde{f}_\mu(x) = f(x) + \mu \text{viol}(x)^2,$$

dove  $\text{viol}(x)$  misura quanto si violano i vincoli (ad esempio  $\text{viol}(x) = \max(x, 0)$  per una disuguaglianza  $x \leq 0$ ). - semplice da implementare; - scelta di  $\mu$  delicata: troppo piccola non rispetta vincoli, troppo grande rende il problema mal condizionato.

(ii) **Barriere (barrier methods)** Per disuguaglianze  $g_i(x) < 0$ :

$$\tilde{f}_\mu(x) = f(x) - \mu \sum_i \log(-g_i(x)).$$

- mantiene iterati nell'interno del dominio ammissibile; - richiede che si inizi da un punto strettamente ammissibile ed “attenzione numerica” vicino al bordo (bisogna evitare di andare vicino ai punti dove il logaritmo diverge).

(iii) **Proiezione (projected methods) – ottima per box** Dopo un passo “libero”  $x - \alpha \nabla f(x)$ , si proietta sul vincolo:

$$x_{n+1} = \Pi_{[a,b]}(x_n - \alpha \nabla f(x_n)).$$

Per vincoli box in 1D, la proiezione è semplicemente il clipping:

$$\Pi_{[a,b]}(x) = \min(\max(x, a), b).$$

**Osservazione cruciale:** con vincoli, la soluzione può essere sul bordo; non ci si deve aspettare  $\nabla f(x^*) = 0$ .

## 8. Esercizi guida: tre casi canonici

1. **Zeri:** Newton che entra in un ciclo o fallisce senza bracketing; bisezione che converge grazie al bracket.
2. **Minimi:** funzione 1D con due bacini; confronto single-start vs multi-start.
3. **Vincolo box:** minimo non vincolato fuori dal box; soluzione vincolata sul bordo; confronto penalita vs proiezione.

Suggerimento operativo: per ogni esercizio, stampare sempre (almeno) residuo/valore funzione, passo e ragione d'uscita.